Dust Growth on Superparticles in AthenaK

Implementation Document

D1: the growth step; D2: diagnostics; the `nsh_init` option of `streaming_strat`

worktree `athenak-worktrees/dust-growth`, branch `dust-growth` (parent `9aaa5639`)

commits `a821cb4d` (growth step, diagnostics, tests) and `e231fc7f` (`nsh_init`, R1 set-up)

October 19, 2026

Abstract

This document explains, file by file and line by line, how dust growth and fragmentation were added to the AthenaK dust module for the NHFP 2027 preliminary result (the streaming instability with live dust growth in a stratified box). The design is the simplest one that makes every existing kernel grow-aware without touching it: each superparticle keeps its mass and carries the size of the grains it represents in its stopping time `IPTS`, which every drag kernel already reads per particle. Once per cycle, before anything else in the time step, an operator-split growth step deposits the dust density, gathers it and the gas velocity at each particle, and advances `IPTS` by the Birnstiel et al. (2012) two-population law for equal-size sticking, reversing sign where the collision speed exceeds the fragmentation velocity. The document derives the law, places the step in the time loop and the task graph, walks through the kernel, lists every other part of the code that sees the changed stopping times (drag, timestep, gamma switch, restart, migration, AMR), and documents the diagnostics, the inputs and the limitations. A companion document (`dust_growth_validation.pdf`) reports the tests.

Contents

1 Scope and how to read this **3**
- 1.1 What this document is . . . 3
- 1.2 Files touched . . . 3

2 Design: why the stopping time carries the size **4**
- 2.1 The representative-particle convention . . . 4
- 2.2 Why the stopping time and not a new field . . . 4
- 2.3 The constant-Stokes convention . . . 4
- 2.4 The `stopping_time_mode` contract . . . 4

3 The physics of the growth law **4**
- 3.1 From sticking to a scale-free law . . . 4
- 3.2 Collision speeds . . . 5
- 3.3 Fragmentation . . . 5
- 3.4 Connection to the two-population growth time . . . 6

4 Where the step sits in a cycle **6**
4.1 The driver loop . . . 6
4.2 The task . . . 6
4.3 Why first, and why once per cycle . . . 7

5 Set-up: `InitGrowth` **7**

6 The step: `GrowthStep` **7**
6.1 Assembling the dust density . . . 8
6.2 The cost . . . 8

7 The kernel: `GrowStoppingTimes` **8**
7.1 Constants (lines 156–165) . . . 8
7.2 Interpolation weights (lines 170–183) . . . 9
7.3 The gather (lines 185–204) . . . 9
7.4 The rate and its limits (lines 205–227) . . . 9
7.5 Counters . . . 10

8 Everything else that sees the stopping time **10**
8.1 The drag kernels (unchanged) . . . 10
8.2 Validation and derived maxima . . . 11
8.3 Migration, restart, AMR . . . 11

9 Diagnostics (D2) **11**
9.1 Particle history (`phst`) . . . 11
9.2 Particle VTK . . . 11

10 The NSH initial condition of `streaming_strat` **12**

11 Input reference **12**

12 Limitations and what comes next **13**

A `src/dust/dust_growth.cpp` **13**

1 Scope and how to read this

1.1 What this document is

It is the reference for the code added on branch `dust-growth`: the growth step (`src/dust/dust_growth.cpp`, task D1 of `HANDOFF_DUST_GROWTH.md`), the diagnostics that make the grown sizes visible (D2), and the NSH initial condition added to `streaming_strat` for the R1 production runs. Section 3 derives the physics; Sections 4–7 follow one cycle through the code; Section 8 lists everything else that reads the stopping time; Sections 9–11 are the diagnostics and parameter reference; Section 12 states what the model does not do. Appendix A reproduces `dust_growth.cpp` in full.

1.2 Files touched

file	change
`src/dust/dust_growth.cpp` (new)	`InitGrowth`, the `GrowthStep` task, the `GrowStoppingTimes` kernel
`src/dust/dust.hpp`	enum `DustGrowth`; growth parameters and counters as members of `DustGasDrag`; task id `id.growth`; the three declarations
`src/dust/dust.cpp`	`InitGrowth(pin)` called from the constructor; `DUST_GROWTH_SUMMARY` in the destructor
`src/dust/dust_tasks.cpp`	`GrowthStep` added to `before_timeintegrator`, with `GammaSwitch` depending on it
`src/CMakeLists.txt`	`dust/dust_growth.cpp` added to the sources
`src/outputs/particle_history.cpp`, `outputs.hpp`	Stokes-number columns of the `phst` output when growth is on
`src/outputs/vtk_prtcl.cpp`	`SCALARS ptaus` in the particle VTK when the stopping times are per particle
`src/pgen/tests/streaming_strat.cpp`	opt-in `<problem>/nsh_init` (Section 10)
`inputs/dust/dust_growth_box.athinput`	the D3 test box
`inputs/dust/streaming_strat_growth_r1.athinput`	the R1 production input
`scripts/analysis/dust_growth_tests.{sh,py}`, `dust_growth_tests_fig.jl`, `si_growth_r1_fig.jl`	test suite, checker, figures
`scripts/cluster/build_growth_h2.slurm`, `si_growth_r1_h2.slurm`	h-2 build and run scripts

Invariant. With `<dust>/growth = none` (the default) the new task returns immediately and nothing else changes: every output of the parent commit is reproduced byte for byte (test (a) of the validation document). The only visible difference is the extra line `growth = none` that AthenaK echoes into the parameter block of restart files.

2 Design: why the stopping time carries the size

2.1 The representative-particle convention

A superparticle stands for a swarm of identical grains. In the representative-particle method (Zsom & Dullemond 2008; the basis of `mcdust`, Drążkowska et al. 2014) the swarm mass is fixed and the grains in it grow: the number of grains per swarm falls as each grain gets heavier. This is what the code does: the particle mass `IPM` never changes, and the grain size changes.

2.2 Why the stopping time and not a new field

AthenaK's dust particles have 17 real fields (`src/particles/particles.cpp`:99): positions, velocities, their RK registers, the recorded drag rates, the stopping time `IPTS` (index 15) and the mass `IPM` (16), declared in `src/athena.hpp`:79. In Epstein drag at fixed gas density the stopping time is proportional to the grain size, so `IPTS` *is* the size up to a constant. The dust module already reads `IPTS` per particle in every drag kernel (listed in Section 8); changing it between steps therefore makes the whole integrator respond to growth with no change to any drag, deposit or solver code. A new size field would have needed a conversion in every one of those kernels.

2.3 The constant-Stokes convention

Shearing-box SI calculations fix the dimensionless stopping time $\tau_s = \Omega t_\mathrm{s}$ rather than the size, i.e. they evaluate the Epstein law at a reference gas density ρ_g0 (the midplane density) instead of the local one. The growth code keeps this convention: the Stokes number of a particle is

$$S = \Omega\, t_\mathrm{s} \qquad (\text{at } \rho_\mathrm{g} = \rho_\mathrm{g0}), \tag{1}$$

so a particle high in the atmosphere has the same t_s as one of the same size at the midplane. Code units are $\rho_\mathrm{g0} = H = \Omega = c_s = 1$ in every input used here, so $S = t_\mathrm{s}$ numerically; the code nevertheless keeps Ω, c_s and ρ_g0 as explicit parameters (Section 11).

2.4 The `stopping_time_mode` contract

The dust module has three modes (`dust.hpp`:46): `species_fixed` (every particle's `IPTS` must equal its species value `taus_s`), `particle_static` (arbitrary positive per-particle values that never change) and `dynamic` (values may change; their global maximum and minimum are refreshed every cycle). Growth changes `IPTS`, so `InitGrowth` refuses to run unless the mode is `dynamic`. The modes are enforced by `RefreshStoppingTimeMaximum` (`dust_newdt.cpp`:34), which aborts if any `IPTS` is non-finite or non-positive, or (in `species_fixed`) differs from its species value; it runs once at start-up in the first two modes and every cycle in `dynamic`.

3 The physics of the growth law

3.1 From sticking to a scale-free law

For Epstein drag the stopping time of a grain of radius a and material density ρ_s is

$$t_\mathrm{s} = \frac{\rho_s a}{\rho_\mathrm{g} v_\mathrm{th}}, \qquad v_\mathrm{th} = \sqrt{8/\pi}\, c_s, \tag{2}$$

so at the reference density

$$S = \sqrt{\frac{\pi}{8}}\,\frac{\rho_s a\,\Omega}{\rho_{\mathrm{g0}} c_s}. \tag{3}$$

Equal-size sticking: a grain of mass $m = \frac{4}{3}\pi\rho_s a^3$ sweeps grains of the same size with cross section $\sigma = \pi(2a)^2$ at relative speed Δv through a dust density ρ_{d}, gaining mass at $\dot{m} = \rho_{\mathrm{d}}\sigma\Delta v$. With $\dot{m} = 4\pi\rho_s a^2\dot{a}$ this is

$$\frac{da}{dt} = \frac{\rho_{\mathrm{d}}\,\Delta v}{\rho_s}, \tag{4}$$

the growth rate of Birnstiel et al. (2012, with sticking efficiency 1). Inserting into (3):

$$\boxed{\frac{dS}{dt} = \sqrt{\frac{\pi}{8}}\,\Omega\,\frac{\rho_{\mathrm{d}}}{\rho_{\mathrm{g0}}}\,\frac{\Delta v}{c_s}} \qquad \Longleftrightarrow \qquad \frac{dt_{\mathrm{s}}}{dt} = \sqrt{\frac{\pi}{8}}\,\frac{\rho_{\mathrm{d}}}{\rho_{\mathrm{g0}}}\,\frac{\Delta v}{c_s}. \tag{5}$$

The grain material density cancels: the law needs no ρ_s and no physical units. The code integrates the second form, so Ω enters only through the turbulent speed below. A physical size for figures follows from (3) with $\Sigma_g = \sqrt{2\pi}\rho_{\mathrm{g0}}H$:

$$a = \frac{2}{\pi}\,\frac{S\,\Sigma_g}{\rho_s}. \tag{6}$$

3.2 Collision speeds

Two sources, added in quadrature:

$$\Delta v^2 = \Delta v_{\mathrm{turb}}^2 + \Delta v_{\mathrm{drift}}^2. \tag{7}$$

Turbulence. $\Delta v_{\mathrm{turb}} = \sqrt{3\alpha_t S}\,c_s$, the intermediate-regime expression of Ormel & Cuzzi (2007) used by Birnstiel et al. (2012). α_t (`growth_alpha`) is a *prescribed* parameter: the box contains no imposed turbulence, so α_t represents unresolved turbulence and is not measured from the flow (Section 12).

Differential drift. $\Delta v_{\mathrm{drift}} = \frac{1}{2}|\mathbf{v}_p - \mathbf{u}_g(\mathbf{x}_p)|$, with $\mathbf{v}_p$ the particle velocity and $\mathbf{u}_g$ the gas velocity interpolated to the particle. For $S \ll 1$ the velocity of a grain relative to the gas is proportional to S, so the partner of half the size moves at half the relative speed and the two differ by half of $|\mathbf{v}_p - \mathbf{u}_g|$. Because $\mathbf{v}_p$ and $\mathbf{u}_g$ are the *actual* local velocities, this term includes radial drift, azimuthal drift and settling as the simulation produces them, including their modification by back-reaction and by the SI. It can be switched off (`growth_drift = false`).

3.3 Fragmentation

Where $\Delta v > v_{\mathrm{frag}}$ (`growth_vfrag`, in units of c_s) the right-hand side of (5) changes sign: grains shrink at the rate they would have grown (the simple form of Vericel et al. 2021). With turbulence alone this caps the Stokes number at

$$S_{\mathrm{frag}} = \frac{v_{\mathrm{frag}}^2}{3\alpha_t c_s^2} \qquad \big(\alpha_t = 10^{-4},\ v_{\mathrm{frag}} = 0.01c_s:\ S_{\mathrm{frag}} = 0.33;\quad \alpha_t = 10^{-3}:\ 0.033\big). \tag{8}$$

Birnstiel et al. (2012) multiply this by a safety factor $f_f \approx 0.37$; the code does not. The particles then dither around the cap with an amplitude of one step's change. Drift alone caps where $\frac{1}{2}|\Delta\mathbf{v}| = v_{\mathrm{frag}}$; for a dust-poor gas $|\Delta\mathbf{v}| \approx 2S\Pi c_s$, giving $S \approx v_{\mathrm{frag}}/(\Pi c_s) = 0.2$ for $v_{\mathrm{frag}} = 0.01c_s$, $\Pi = 0.05$. Back-reaction reduces the relative drift by about $1/(1+\epsilon)$ in a dust-rich layer, which raises the drift cap there.

3.4 Connection to the two-population growth time

In a settled Gaussian layer of thickness $H_d = H\sqrt{\alpha_t/(\alpha_t + S)}$ the midplane dust density is $\rho_\mathrm{d} = Z\rho_{\mathrm{g}0}H/H_d$. With turbulent collisions and $S \gg \alpha_t$, (5) gives

$$\frac{dS}{dt} = \sqrt{\frac{3\pi}{8}}\, Z\,\Omega\, S \quad \Rightarrow \quad t_\mathrm{grow} = \frac{S}{dS/dt} \approx \frac{0.92}{Z\Omega}, \tag{9}$$

the size e-folding time of the Birnstiel et al. (2012) two-population model. The superparticle law reproduces it where the layer is in that equilibrium, and does something more general where it is not (each particle sees its own local ρ_d and Δv).

4 Where the step sits in a cycle

4.1 The driver loop

`Driver::Execute` (`src/driver/driver.cpp`:533) runs, every cycle:

1. the task list `before_timeintegrator` (stage 0, line 565);
2. for each RK stage: `before_stagen`, the Poisson solve (if `<gravity>`), `stagen`, `after_stagen`;
3. `after_timeintegrator`;
4. $t \to t + \Delta t$, `ncycle++`;
5. outputs (`LoadOutputData`/`WriteOutputFile`, line 628);
6. adaptive mesh refinement (line 634);
7. `Mesh::NewTimeStep` (line 636): the Δt of the *next* cycle.

The growth step is the first task of item 1, so it acts on the state that was just written to disk and uses the Δt that the coming cycle will take.

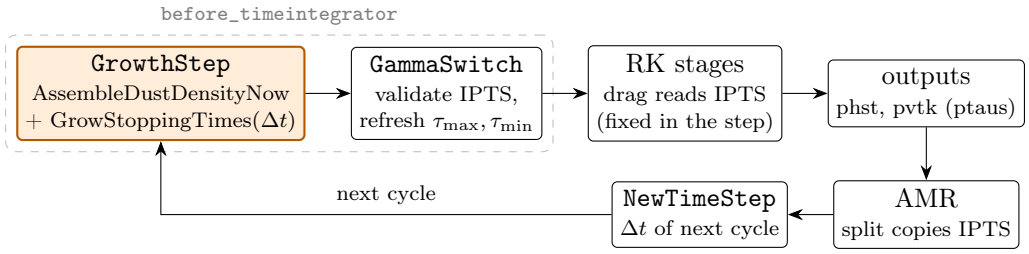


Figure 1: One cycle. The new task (orange) runs first; `GammaSwitch` depends on it.

4.2 The task

`dust_tasks.cpp`:44–48:

```
id.growth  = tl["before_timeintegrator"]->AddTask(&DustGasDrag::GrowthStep,this,none);
id.gswitch = tl["before_timeintegrator"]->AddTask(&DustGasDrag::GammaSwitch,this,
                                                  id.growth);
```

Before this change `GammaSwitch` depended on nothing. The dependency guarantees the order growth → validation; the task is added unconditionally and returns immediately when `growth = none`.

4.3 Why first, and why once per cycle

- **The stopping time must not change inside a step.** The imex2+ integrator records drag rates at one implicit stage and applies them at the next (the `a_twid` history term, `IPRX`–`IPRZ`); the hybrid PC2 and split-BE paths also assume one t_s per step. All of this is consumed within one cycle, so a change between cycles is safe and a change between stages would not be.
- **The derived quantities must be refreshed after the change.** `GammaSwitch` calls `RefreshStoppingTimeMaximum` first thing (in `dynamic` mode), which validates the new values and recomputes $\tau_{\max}$ (used by the imex2+ γ switch, Krapp et al. 2024 eq. 18) and $\tau_{\min}$ (used by the hybrid coupling to choose PC2 or split-BE). Running growth before it means both are always computed from the stopping times the step will actually use.
- **Operator splitting.** The step is a first-order (Lie) split: growth over Δt from the start-of-cycle state, then the full dynamics over Δt with fixed t_s. The splitting error is $O(\Delta t/t_{\rm grow})$ per unit time; in the production runs $\Delta t \approx 2\times10^{-4}\Omega^{-1}$ against $t_{\rm grow} \gtrsim 10\,\Omega^{-1}$. Test (b) of the validation document measures the first-order convergence.

5 Set-up: `InitGrowth`

Called from the `DustGasDrag` constructor (`dust.cpp`:453) after the stopping-time mode, the EOS and the shearing-box parameters have been read, because it needs all three. In order (`dust_growth.cpp`:58–117):

1. `<dust>/growth` is `none` (return; every growth member keeps its default from `dust.hpp`) or `monodisperse`; anything else is a fatal error.
2. `stopping_time_mode` must be `dynamic` (fatal otherwise).
3. `growth_alpha` and `growth_vfrag` are read with `GetReal`, so they are *required*: there is no sensible default for either.
4. `growth_smin` (10^{-5}), `growth_smax` (1), `growth_maxfrac` (0.05), `growth_drift` (true), `growth_rho0` (1) have defaults; `growth_omega` defaults to `<shearing_box>/omega0` in a shearing box and to 1 otherwise; `growth_cs` defaults to the isothermal sound speed and is required for an ideal gas (whose sound speed varies).
5. Validation: $\alpha_t \geq 0$, $v_{\rm frag} > 0$, $0 < S_{\min} < S_{\max}$, positive cap, $\rho_{\rm g0}$, Ω, c_s.
6. Rank 0 prints one line with the law, the parameters and $S_{\rm frag}$.

6 The step: `GrowthStep`

```
125 TaskStatus DustGasDrag::GrowthStep(Driver *pdrive, int stage) {
126   if (growth == DustGrowth::none) {return TaskStatus::complete;}
127   // collective (ghost exchanges): every rank, every cycle, even without particles
128   AssembleDustDensityNow();
129   GrowStoppingTimes(pmy_pack->pmesh->dt);
130   return TaskStatus::complete;
131 }
```

6.1 Assembling the dust density

The kernel needs $\rho_{\rm d}$ at each particle. Particle-mesh interpolation with the TSC kernel reads a $3 \times 3 \times 3$ block of cells around the particle, which reaches one cell into the ghost zone for a particle near a MeshBlock edge, so the density must be complete in the active zone *and* correct in the first ghost layer. `AssembleDustDensityNow` (`dust_gravity.cpp`:288), written for the self-gravity source and the `dust_dpm` output, does exactly this, synchronously:

1. `DepositMass` (line 78): zero `rho_dust`; scatter $m_p W(\mathbf{x} - \mathbf{x}_p)/V$ with the module's deposit kernel (TSC by default) into the active cells and the ghost cells. On a refined mesh every block deposits with the kernel of the finest level present, through a fine image that is then volume-averaged onto the block (`DepositStencil`, `RestrictImage`), so the density is one consistent field across level jumps.

2. Additive exchange (`pbval_rd`): each block sends the mass it deposited into its ghost cells to the neighbour that owns those cells, which adds it to its active cells. After this the active zone holds the complete density.

3. Shear fold (`FoldShearDeposit`): across a shear-periodic x_1 face the neighbour's cells are offset in y by $q\Omega L_x t$, so the ghost deposits are remapped conservatively in y before being added.

4. Copy exchange (`pbval_rc`) and its shear remap (`psbox_rc`): now that every active cell is complete, the ghost layer is overwritten with the neighbours' active values (prolongated next to coarser neighbours on a refined mesh).

Every step is collective, so `GrowthStep` calls it on every rank, every cycle, including ranks without particles; the ordering of collectives is the same on all ranks because `before_timeintegrator` executes its tasks in a fixed order.

At physical (non-periodic) faces nothing fills the ghost layer from outside; there it keeps the block's own ghost deposits. That only matters for particles within a cell of such a face (never the case in the periodic-z R1 box).

6.2 The cost

One extra deposit and three exchanges per cycle. Measured: $\approx 1\%$ in the YJ14 box on 4 Mac ranks, and $\approx 7.5\%$ in the R1 box on 64 h-2 ranks, where the comparison also includes the per-cycle `MPI_Allreduce`s of `dynamic` mode.

7 The kernel: `GrowStoppingTimes`

One Kokkos `parallel_reduce` over the particles of the rank; the three reduction variables count how often the cap, the clamp and fragmentation act.

7.1 Constants (lines 156–165)

```
156   const Real coef = std::sqrt(M_PI/8.0)/(growth_rho0*growth_cs);  // d t_s/dt per rho_d dv
157   const Real turb = 3.0*growth_alpha*growth_omega*SQR(growth_cs);  // dv_turb^2 per t_s
158   const Real vfrag = growth_vfrag*growth_cs;
159   const Real tsmin = growth_smin/growth_omega, tsmax = growth_smax/growth_omega;
```

`coef` is the prefactor of the t_s form of (5), so that $\dot{t_s} =$ `coef` $\cdot \rho_d \cdot \Delta v$. `turb` is $3\alpha_t \Omega c_s^2$, so that $\Delta v_{\rm turb}^2 =$ `turb` $\cdot t_s = 3\alpha_t S c_s^2$. The Stokes limits are converted to stopping-time limits once. Class members are copied into local `const`s because a device lambda cannot capture `this`.

7.2 Interpolation weights (lines 170–183)

`PMWeights` (`dust.hpp`:149) is the same helper every drag kernel uses: for a coordinate it returns the index of the nearest cell (`ip`) and three weights for cells `ip-1`, `ip`, `ip+1` (NGP, CIC or TSC by `<dust>/deposit`). Gathers always use the block's own cells (on a refined mesh, unlike deposits, there is no fine image), exactly as in `GatherKickPMBR`. In 2D only x_1, x_2 are weighted.

7.3 The gather (lines 185–204)

```
185   Real rd = 0.0, ux = 0.0, uy = 0.0, uz = 0.0;
186   ...
187         rd += w*rhod(m,0,kk,jj,ii);
188         ux += w*w0(m,IVX,kk,jj,ii);
189         Real vy = w0(m,IVY,kk,jj,ii);
190         if (full) {
191           vy += qo*(mbsize.d_view(m).x1min + (static_cast<Real>(ii - is) + 0.5)*dx1);
192         }
193         uy += w*vy;
194         uz += w*w0(m,IVZ,kk,jj,ii);
```

The dust density comes from `rho_dust`, just assembled. It contains every particle's mass, including the particle's own swarm (spread by its kernel). That is the representative-particle convention (Zsom & Dullemond 2008): a representative grain collides with grains of every swarm in its neighbourhood, its own included.

The gas velocity comes from the primitive array `w0`. At the start of a cycle `w0` is valid in the active zone and in all ghost cells: the previous stage ended with the boundary exchange, the physical boundary conditions and `ConToPrim` over the whole array, and the first stage of the coming cycle needs those ghosts for its fluxes anyway. The drag kernels gather a different field (the provisional velocity u^* of the implicit solve), which does not exist yet at this point of the cycle; `w0` is the end-of-step gas velocity.

Frames. Particle velocities are always shear-relative. With orbital advection (FARGO) the gas velocities are too. Without it (`gas_full_frame`, 3D only) the gas v_y includes the background shear $-q\Omega x$, and the kernel adds back $+q\Omega x$ at each stencil cell's centre: the same shift `ShiftGasFrame` (`dust_frame.cpp`) applies to the gas momenta for the drag tasks. Because TSC weights reproduce linear functions exactly ($\sum w_a = 1$, $\sum w_a x_a = x_p$), the interpolated shift equals $q\Omega x_p$ to round-off, so the frame correction is exact. In the 2D r–z shearing box the azimuthal components are `IVZ` and `IPVZ`, and they are paired consistently by component index.

7.4 The rate and its limits (lines 205–227)

```
205   Real ts = pr(IPTS,p);
206   Real dv2 = turb*ts;
207   if (drift) {
208     dv2 += 0.25*(SQR(pr(IPVX,p) - ux) + SQR(pr(IPVY,p) - uy) + SQR(pr(IPVZ,p) - uz));
209   }
```

```
    Real dv = sqrt(dv2);
    Real rate = coef*fmax(rd, 0.0)*dv;
    if (dv > vfrag) {
      rate = -rate;
      frag += 1;
    }
    Real dts = rate*dt;
    Real lim = maxfrac*ts;
    if (fabs(dts) > lim) {
      dts = (dts > 0.0) ? lim : -lim;
      cap += 1;
    }
    Real tsn = ts + dts;
    if (tsn < tsmin || tsn > tsmax) {
      tsn = fmin(fmax(tsn, tsmin), tsmax);
      clamp += 1;
    }
    pr(IPTS,p) = tsn;
```

- $\Delta v^2 = 3\alpha_t Sc_s^2 + \frac{1}{4}|\mathbf{v}_p - \mathbf{u}_g|^2$; the drift term uses all three components.
- `fmax(rd,0)` guards against a negative interpolated density (not possible with TSC weights and positive masses, but cheap).
- Forward Euler over Δt: $t_\mathrm{s} \to t_\mathrm{s} + \dot{t_\mathrm{s}}\Delta t$.
- **Cap** (`growth_maxfrac`): no particle changes its t_s by more than this fraction in one cycle. In normal operation it never binds (the per-step change is $\sim \Delta t/t_\mathrm{grow} \sim 10^{-5}$); it exists so that a pathological input (huge α_t or ρ_d) cannot produce a negative or exploding t_s in one step. When it binds, the result depends on Δt, so the count is reported.
- **Clamp** (`growth_smin`, `growth_smax`): a hard floor and ceiling on S. A particle sitting at a limit is counted every cycle.
- The new value overwrites `IPTS` in place; nothing else about the particle changes.

7.5 Counters

The three reduction results are added to rank-local `unsigned long long` totals (`growth_capped`, `growth_clamped`, `growth_frag`); the destructor sums them over ranks and rank 0 prints

```
# DUST_GROWTH_SUMMARY steps=N capped=.. clamped=.. fragmenting=.. (particle-steps; ...)
```

(`steps` is the number of growth steps of rank 0).

8 Everything else that sees the stopping time

8.1 The drag kernels (unchanged)

where	use of `IPTS`
`dust_push.cpp`:186, 224–226	hybrid PC2: implicit-midpoint factor $\lambda = \frac{1}{2}\Delta t/t_\mathrm{s}$ and the drag impulse $\Delta t(u-v)/t_\mathrm{s}$

`dust_push.cpp`:306–308	hybrid split-BE: $c_j = \Delta t/(t_\mathrm{s} + \Delta t)$ and the deposited rate $m/(t_\mathrm{s}V)$
`dust_pm.cpp`:114	imex2+ deposit of $Q = \sum m/(t_\mathrm{s}V)$ for the per-cell solve
`dust_pm.cpp`:180, 403	imex2+ gather–kick: $c_j = a\Delta t/(t_\mathrm{s} + a\Delta t)$
`dust_solver.cpp`:177, 504, 558	coupled solvers (PCG, adaptive)

All of them read the value per particle and per stage, so they use the grown value from the cycle after it was set. None constrains the timestep: AthenaK's dust integrators are stable for any $\Delta t/t_\mathrm{s}$, so growth to large t_s or a floor at small t_s never changes Δt.

8.2 Validation and derived maxima

`RefreshStoppingTimeMaximum` (`dust_newdt.cpp`:34), called by `GammaSwitch` right after the growth step, aborts the run if any `IPTS` is non-finite or ≤ 0 (so a bug in growth cannot silently corrupt a run), then sets τ_max and τ_min with two global reductions.

8.3 Migration, restart, AMR

- **Migration** between MeshBlocks and ranks packs all `nrdata` real fields of a particle (`bvals/bvals_part.cpp`:577–589), so `IPTS` travels with the particle.
- **Restart** files write and read all `nrdata` fields (`outputs/restart.cpp`:273, 645); a restarted run continues bitwise (test (e)).
- **AMR splitting**: when a block refines with `<particles>/split_on_refine`, each particle becomes 2^d children that copy all real fields of the parent and get $1/2^d$ of its mass (`particles/particles_amr.cpp`:133–150); children carry the parent's grown `IPTS` (test (e), split check).

9 Diagnostics (D2)

9.1 Particle history (`phst`)

When growth is on, `ParticleHistoryOutput` appends six columns after the per-species ones: `St_mean` (mass-weighted mean of $S = \Omega t_\mathrm{s}$), `St_max`, `St_min`, and the mass fractions `fm_St>0.01`, `fm_St>0.03`, `fm_St>0.1`. One more `parallel_reduce` with four sums, a max and a min; sums are reduced over ranks with `MPI_SUM`, the extrema with `MPI_MAX`/`MPI_MIN`. With growth off the file is unchanged.

9.2 Particle VTK

`ParticleVTKOutput` writes a `SCALARS ptaus` block (after the integer scalars, before `VECTORS pvel`) whenever `stopping_time_mode` is not `species_fixed`. In `species_fixed` runs the stopping time follows from `pspecies`, and the historical schema and I/O volume are kept. Readers that locate blocks by keyword (`baehr_table1.py`, `dust_growth_tests.py`, `si_growth_r1_fig.jl`) are unaffected by the extra block.

10 The NSH initial condition of `streaming_strat`

For R1 the stratified-SI generator gained `<problem>/nsh_init` (default false; with it false the generator's output is bitwise unchanged). Lim et al. (2025, ApJ 981, 160) start gas and particles at the Nakagawa et al. (1986) drift equilibrium for the local dust-to-gas ratio; without it a dust-rich midplane starts out of force balance and sloshes radially.

For one species with $\tau = \Omega t_{\rm s}$ and the radial forcing $a_x = 2\Omega\eta v_K$ on the gas (`etavk` $= \eta v_K$), the steady shear-relative velocities solve

$$0 = 2\Omega u_y + a_x + \epsilon(v_x - u_x)/t_{\rm s}, \qquad 0 = -(2-q)\Omega u_x + \epsilon(v_y - u_y)/t_{\rm s}, \tag{10}$$

$$0 = 2\Omega v_y - (v_x - u_x)/t_{\rm s}, \qquad 0 = -(2-q)\Omega v_x - (v_y - u_y)/t_{\rm s}, \tag{11}$$

whose solution for general q is, with $D = (1+\epsilon)^2 + 2(2-q)\tau^2$,

$$v_x = -\frac{2\tau\eta v_K}{D}, \quad v_y = -\frac{(1+\epsilon)\eta v_K}{D}, \quad u_x = \frac{2\epsilon\tau\eta v_K}{D}, \quad u_y = -\frac{(1+\epsilon+2(2-q)\tau^2)\eta v_K}{D} \tag{12}$$

(the familiar Keplerian form for $q = 3/2$). The local ratio is $\epsilon(z) = \rho_p(z)/\rho_{\rm g}(z)$ with $\rho_{\rm g} = \rho_{\rm g0}e^{-z^2/2H^2}$ and, for the Gaussian layer of width h_z truncated to the box (the generator's particle placement), $\rho_p(z) = Z\Sigma_g e^{-z^2/2h_z^2}/(\sqrt{2\pi}h_z f)$, where f is the fraction of the Gaussian inside the box ($\rho_p = Z\Sigma_g/L_z$ for a uniform layer). Gas cells use ϵ at their centre, particles at their own height; vertical velocities are zero; `gas_vy0` is ignored. Only one species is supported (fatal otherwise). With orbital advection off, the background shear is added to the gas momentum as before.

11 Input reference

parameter (`<dust>`)	default	meaning
`growth`	`none`	`none` or `monodisperse`
`stopping_time_mode`	`species_fixed`	must be `dynamic` with growth
`growth_alpha`	required	α_t of $\Delta v_{\rm turb} = \sqrt{3\alpha_t S}c_s$
`growth_vfrag`	required	$v_{\rm frag}/c_s$
`growth_smin`, `growth_smax`	10^{-5}, 1	clamp on S
`growth_maxfrac`	0.05	largest fractional change of $t_{\rm s}$ per cycle
`growth_drift`	true	include $\frac{1}{2}\lvert\mathbf{v}_p - \mathbf{u}_g\rvert$
`growth_rho0`	1	$\rho_{\rm g0}$ of the Stokes number and of (5)
`growth_omega`	`omega0` (or 1)	Ω in $S = \Omega t_{\rm s}$ and in $\Delta v_{\rm turb}$
`growth_cs`	`iso_sound_speed`	c_s (required for an ideal gas)
`<problem>/nsh_init`	false	`streaming_strat`: NSH velocities for the local $\epsilon(z)$

Initial stopping times come from `<dust>/taus_s` through the problem generator as before; growth starts from whatever `IPTS` the generator wrote.

12 Limitations and what comes next

- **Locally monodisperse.** Each particle grows as if every grain within its kernel had its own size: the partner density is the total $\rho_{\rm d}$ and the relative speed is that of equal (turbulence) or half-size (drift) partners. Particles of different sizes in the same cell do not collide with each other's actual sizes. A Monte Carlo collision step between superparticles (the `mcdust` approach) is the Year-1 deliverable that removes this.
- **Prescribed α_t.** $\Delta v_{\rm turb}$ uses a parameter, while the box has only SI-driven turbulence. The dust layer is therefore thinner, and $\rho_{\rm d}$ higher, than in a real disk with the same α_t, which makes growth faster. A self-consistent alternative (task D4) is Δv from the resolved local particle velocity dispersion.
- **Constant-Stokes convention.** $t_{\rm s}$ is evaluated at $\rho_{\rm g0}$, not at the local gas density, as in SI calculations without growth.
- **Sharp fragmentation switch** without the f_f safety factor or a smooth transition; particles at the cap dither by one step's change.
- **First-order splitting** (Section 4); the error is negligible at production timesteps but visible in the deliberately coarse test (b).
- **Cost:** one extra deposit and exchange per cycle (1–8% measured). A `growth_interval` that accumulates Δt over several cycles would reduce it if needed.

A `src/dust/dust_growth.cpp`

```
//========================================================================================
// AthenaXXX astrophysical plasma code
// Copyright(C) 2020 James M. Stone <jmstone@ias.edu> and the Athena code team
// Licensed under the 3-clause BSD License (the "LICENSE")
//========================================================================================
//! \file dust_growth.cpp
//! \brief Dust growth and fragmentation carried by the per-particle stopping time.
//!
//! Representative-particle convention: the mass IPM of a superparticle is fixed, and the
//! size of the grains it represents is carried by its stopping time IPTS (one size per
//! particle).  Under the constant-Stokes convention of shearing-box SI calculations, the
//! Stokes number S = Omega t_s is evaluated at the reference (midplane) gas density
//! rho_g0.  Epstein drag, t_s = rho_s a/(rho_g0 v_th) with v_th = sqrt(8/pi) c_s, and
//! equal-size sticking, da/dt = rho_d dv/rho_s (Birnstiel et al. 2012), give a law in
//! which the grain material density cancels:
//!     d t_s/dt = sqrt(pi/8) (rho_d/rho_g0) (dv/c_s)      [dS/dt = Omega d t_s/dt]
//! with rho_d the local PM dust density (all species) gathered at the particle, and
//!     dv^2 = dv_turb^2 + dv_drift^2,
//!     dv_turb  = sqrt(3 alpha_t S) c_s                (Ormel & Cuzzi 2007, intermediate)
//!     dv_drift = |v_p - u_g(x_p)|/2                   (vs a half-size partner)
//! where u_g is the gas velocity gathered with the drag kernel's stencil.  Where
//! dv > v_frag the grains fragment and shrink at the same rate (Vericel et al. 2021);
//! the turbulent term alone then caps S near S_frag = v_frag^2/(3 alpha_t c_s^2).
//!
//! The step is operator split: once per cycle, in "before_timeintegrator" before
//! GammaSwitch revalidates the stopping times and refreshes taus_max/taus_min, so IPTS is
//! never changed inside a step (the imex2+ drag history is consumed within one cycle).
```

```
//! The fractional change of t_s per cycle is capped at <dust>/growth_maxfrac, and S is
//! clamped to [growth_smin, growth_smax]; cumulative counts of particle-steps where
//! these limits bind are printed at the end of the run (DUST_GROWTH_SUMMARY).
//!
//! Requires <dust>/stopping_time_mode = dynamic.  Caveat: alpha_t must be consistent with
//! the box's actual turbulence; in an SI-only box (no imposed turbulence) a large alpha_t
//! makes the growth artificially fast.

#include <cmath>
#include <iomanip>
#include <iostream>
#include <string>

#include "athena.hpp"
#include "globals.hpp"
#include "parameter_input.hpp"
#include "mesh/mesh.hpp"
#include "driver/driver.hpp"
#include "eos/eos.hpp"
#include "hydro/hydro.hpp"
#include "particles/particles.hpp"
#include "dust.hpp"

namespace dust {

//----------------------------------------------------------------------------------------
//! \fn DustGasDrag::InitGrowth
//! \brief Parses and validates the <dust>/growth* parameters (called by the constructor
//! once the stopping-time mode, EOS and shearing-box parameters are known)

void DustGasDrag::InitGrowth(ParameterInput *pin) {
  std::string mode = pin->GetOrAddString("dust", "growth", "none");
  if (mode.compare("none") == 0) {
    growth = DustGrowth::none;
    return;
  } else if (mode.compare("monodisperse") == 0) {
    growth = DustGrowth::monodisperse;
  } else {
    std::cout << "### FATAL ERROR in " << __FILE__ << " at line " << __LINE__ << std::endl
              << "<dust>/growth = '" << mode << "' not recognized (must be none or "
              << "monodisperse)" << std::endl;
    std::exit(EXIT_FAILURE);
  }
  if (stopping_time_mode != DustStoppingTimeMode::dynamic) {
    std::cout << "### FATAL ERROR in " << __FILE__ << " at line " << __LINE__ << std::endl
              << "<dust>/growth = " << mode << " changes the particle stopping times and "
              << "requires <dust>/stopping_time_mode = dynamic" << std::endl;
    std::exit(EXIT_FAILURE);
  }
  growth_alpha   = pin->GetReal("dust", "growth_alpha");
  growth_vfrag   = pin->GetReal("dust", "growth_vfrag");     // in units of c_s
  growth_smin    = pin->GetOrAddReal("dust", "growth_smin", 1.0e-5);
  growth_smax    = pin->GetOrAddReal("dust", "growth_smax", 1.0);
  growth_maxfrac = pin->GetOrAddReal("dust", "growth_maxfrac", 0.05);
  growth_drift   = pin->GetOrAddBoolean("dust", "growth_drift", true);
  // reference gas density of the Stokes number and the units of dv
  growth_rho0    = pin->GetOrAddReal("dust", "growth_rho0", 1.0);
  growth_omega   = pin->GetOrAddReal("dust", "growth_omega",
                                     is_shearing_box ? omega0 : 1.0);
```

```
  if (gas_ideal) {
    growth_cs = pin->GetReal("dust", "growth_cs");
  } else {
    growth_cs = pin->GetOrAddReal("dust", "growth_cs",
                                  pmy_pack->phydro->peos->eos_data.iso_cs);
  }
  if (!(growth_alpha >= 0.0) || !(growth_vfrag > 0.0) || !(growth_smin > 0.0) ||
      !(growth_smax > growth_smin) || !(growth_maxfrac > 0.0) ||
      !(growth_rho0 > 0.0) || !(growth_omega > 0.0) || !(growth_cs > 0.0)) {
    std::cout << "### FATAL ERROR in " << __FILE__ << " at line " << __LINE__ << std::endl
              << "Invalid <dust>/growth parameters: need growth_alpha >= 0, "
              << "growth_vfrag > 0, 0 < growth_smin < growth_smax, growth_maxfrac > 0, "
              << "and positive growth_rho0, growth_omega, growth_cs" << std::endl;
    std::exit(EXIT_FAILURE);
  }
  if (global_variable::my_rank == 0) {
    std::cout << std::setprecision(6)
              << "# dust: growth = monodisperse: d t_s/dt = sqrt(pi/8) (rho_d/rho_g0) "
              << "dv/c_s, dv^2 = 3 alpha_t S c_s^2"
              << (growth_drift ? " + |v_p - u_g|^2/4" : "")
              << "; alpha_t = " << growth_alpha << ", v_frag = " << growth_vfrag
              << " c_s";
    if (growth_alpha > 0.0) {
      std::cout << " (turbulent cap S_frag = "
                << SQR(growth_vfrag)/(3.0*growth_alpha) << ")";
    }
    std::cout << ", S in [" << growth_smin << ", " << growth_smax
              << "], max fractional change " << growth_maxfrac << " per cycle"
              << " (rho_g0 = " << growth_rho0 << ", Omega = " << growth_omega
              << ", c_s = " << growth_cs << ")" << std::endl;
  }
}

//----------------------------------------------------------------------------------------
//! \fn DustGasDrag::GrowthStep
//! \brief "before_timeintegrator" task: assemble the PM dust density with its ghost
//! layer, then advance every particle's stopping time over the coming step dt

TaskStatus DustGasDrag::GrowthStep(Driver *pdrive, int stage) {
  if (growth == DustGrowth::none) {return TaskStatus::complete;}
  // collective (ghost exchanges): every rank, every cycle, even without particles
  AssembleDustDensityNow();
  GrowStoppingTimes(pmy_pack->pmesh->dt);
  return TaskStatus::complete;
}

//----------------------------------------------------------------------------------------
//! \fn DustGasDrag::GrowStoppingTimes
//! \brief The growth kernel: gathers rho_d (rho_dust, ghost-filled) and the gas velocity
//! (w0, valid in the ghosts at the start of a cycle) with the drag kernels' PM stencil

void DustGasDrag::GrowStoppingTimes(Real dt) {
  ++growth_steps;
  particles::Particles *ppar = pmy_pack->ppart;
  int npart = ppar->nprtcl_thispack;
  if (npart == 0) {return;}

  auto &pr = ppar->prtcl_rdata;
  auto &pi = ppar->prtcl_idata;
```

```
  auto &w0 = pmy_pack->phydro->w0;
  auto &rhod = rho_dust;
  auto &indcs = pmy_pack->pmesh->mb_indcs;
  int is = indcs.is, js = indcs.js, ks = indcs.ks;
  int nx1 = indcs.nx1, nx2 = indcs.nx2, nx3 = indcs.nx3;
  bool three_d = pmy_pack->pmesh->three_d;
  auto &mbsize = pmy_pack->pmb->mb_size;
  auto gids = pmy_pack->gids;
  int scheme = static_cast<int>(deposit);

  const Real coef = std::sqrt(M_PI/8.0)/(growth_rho0*growth_cs);  // d t_s/dt per rho_d dv
  const Real turb = 3.0*growth_alpha*growth_omega*SQR(growth_cs);  // dv_turb^2 per t_s
  const Real vfrag = growth_vfrag*growth_cs;
  const Real tsmin = growth_smin/growth_omega, tsmax = growth_smax/growth_omega;
  const Real maxfrac = growth_maxfrac;
  const bool drift = growth_drift;
  // 3D shearing box without orbital advection: w0 carries the background shear, the
  // particle velocities do not (the shift of ShiftGasFrame, at each stencil cell)
  const bool full = gas_full_frame;
  const Real qo = qshear*omega0;

  int ncap = 0, nclamp = 0, nfrag = 0;
  Kokkos::parallel_reduce("dust_growth", Kokkos::RangePolicy<>(DevExeSpace(), 0, npart),
  KOKKOS_LAMBDA(const int p, int &cap, int &clamp, int &frag) {
    int m = pi(PGID,p) - gids;
    int ip, jp, kp;
    Real wx[3], wy[3], wz[3];
    PMWeights(pr(IPX,p), mbsize.d_view(m).x1min, mbsize.d_view(m).x1max, nx1, is,
              scheme, ip, wx);
    PMWeights(pr(IPY,p), mbsize.d_view(m).x2min, mbsize.d_view(m).x2max, nx2, js,
              scheme, jp, wy);
    if (three_d) {
      PMWeights(pr(IPZ,p), mbsize.d_view(m).x3min, mbsize.d_view(m).x3max, nx3, ks,
                scheme, kp, wz);
    } else {
      kp = ks;
      wz[0] = 0.0; wz[1] = 1.0; wz[2] = 0.0;
    }
    Real dx1 = (mbsize.d_view(m).x1max - mbsize.d_view(m).x1min)/static_cast<Real>(nx1);
    Real rd = 0.0, ux = 0.0, uy = 0.0, uz = 0.0;
    int clo = three_d ? 0 : 1, chi = three_d ? 2 : 1;
    for (int c=clo; c<=chi; ++c) {
      for (int b=0; b<3; ++b) {
        Real wcb = wz[c]*wy[b];
        if (wcb == 0.0) continue;
        for (int a=0; a<3; ++a) {
          Real w = wcb*wx[a];
          int kk = kp+c-1, jj = jp+b-1, ii = ip+a-1;
          rd += w*rhod(m,0,kk,jj,ii);
          ux += w*w0(m,IVX,kk,jj,ii);
          Real vy = w0(m,IVY,kk,jj,ii);
          if (full) {
            vy += qo*(mbsize.d_view(m).x1min + (static_cast<Real>(ii - is) + 0.5)*dx1);
          }
          uy += w*vy;
          uz += w*w0(m,IVZ,kk,jj,ii);
        }
      }
    }
```

```
    Real ts = pr(IPTS,p);
    Real dv2 = turb*ts;
    if (drift) {
      dv2 += 0.25*(SQR(pr(IPVX,p) - ux) + SQR(pr(IPVY,p) - uy) + SQR(pr(IPVZ,p) - uz));
    }
    Real dv = sqrt(dv2);
    Real rate = coef*fmax(rd, 0.0)*dv;
    if (dv > vfrag) {
      rate = -rate;
      frag += 1;
    }
    Real dts = rate*dt;
    Real lim = maxfrac*ts;
    if (fabs(dts) > lim) {
      dts = (dts > 0.0) ? lim : -lim;
      cap += 1;
    }
    Real tsn = ts + dts;
    if (tsn < tsmin || tsn > tsmax) {
      tsn = fmin(fmax(tsn, tsmin), tsmax);
      clamp += 1;
    }
    pr(IPTS,p) = tsn;
  }, Kokkos::Sum<int>(ncap), Kokkos::Sum<int>(nclamp), Kokkos::Sum<int>(nfrag));
  growth_capped += static_cast<unsigned long long>(ncap);
  growth_clamped += static_cast<unsigned long long>(nclamp);
  growth_frag += static_cast<unsigned long long>(nfrag);
}

} // namespace dust
```